

SESSION – 8

Maximum Subarray

```
class Solution {
    public int maxSubArray(int[] nums) {
        int maxSum = Integer.MIN_VALUE;
        int currentSum = 0;

        for (int i = 0; i < nums.length; i++) {
            currentSum += nums[i];

            if (currentSum > maxSum) {
                maxSum = currentSum;
            }

            if (currentSum < 0) {
                currentSum = 0;
            }
        }

        return maxSum;
    }
}
```

The screenshot shows a coding platform interface for the 'Maximum Subarray' problem (LeetCode 53). The interface is divided into three main sections:

- Left Pane (Problem Description):** Displays the problem title '53. Maximum Subarray' with a 'Solved' status. It includes the problem description: 'Given an integer array `nums`, find the subarray with the largest sum, and return its sum.' It also provides three examples:
 - Example 1:** Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`, Output: `6`, Explanation: The subarray `[4,-1,2,1]` has the largest sum `6`.
 - Example 2:** Input: `nums = [1]`, Output: `1`, Explanation: The subarray `[1]` has the largest sum `1`.
 - Example 3:** Input: `nums = [5,4,-1,7,8]`, Output: `23`, Explanation: The subarray `[5,4,-1,7,8]` has the largest sum `23`.
- Middle Pane (Test Results):** Shows the test results for three cases. All three cases are marked as 'Accepted'. The runtime is `0 ms`. The input for Case 1 is `nums = [-2,1,-3,4,-1,2,1,-5,4]`, the output is `6`, and the expected output is `6`.
- Right Pane (Code):** Displays the Java code for the solution, which is the same code as shown in the previous block.